

TODO:

- Ausfüllrätsel
- Entscheidbarkeit
- Halteproblem
- Liste der NP-Vollständigen Probleme
- TM Berechnungsgeschichte
- extras
 - CNF konstruieren
 - Minimalautomat
 - Mengenoperationen
 - DEA/NEA/TM/NTM
- Beispiele:
 - Ist Sprache x regulär? → Pumping Lemma
 - Ist Sprache x kontextfrei? → Pumping Lemma
 - Kann eine NTM in polynomieller Zeit entscheiden, ob ein Ausfüllrätsel eine Lösung hat? → Satz von Rice / P Verifizierer / Reduktion
 - Ist Grammatik eindeutig? (FS_2026)5))
 - Parse-tree finden (FS_2022)5))
- Prüfungsaufgaben:
 - HS_2021)4)
 - FS_2022)4)

Begriff	Bedeutung
Regulär	Es gibt einen DEA A, der L akzeptiert, also $L(A) = L$
Kontextfrei	L kann nur von einem ND PDA erkannt werden

VORGEHEN ZU AUFGABENTYPEN

Ist Sprache regulär?	Ja: DEA / NEA / RegEx Nein: Pumping Lemma
Ist Sprache kontextfrei?	Ja: Stackautomat / CNF / BNF Nein: Pumping Lemma (CFL)
Turing-Maschinen	TODO: Berechnungsgeschichte
Ist eine Sprache Turing-Vollständig?	TODO: Ja: TM-Simulation Nein: Halteproblem
Kann Problem von einem Computer gelöst werden?	TODO: Reduktion auf Halteproblem

Kann ein Programm entscheiden, ob zulässige Inputs eine Eigenschaft erfüllen?	TODO: Satz von Rice
---	-------------------------------

Kann eine nichtdeterministische Maschine ... entscheiden?	TODO: NP-Verifizierer
---	---------------------------------

Warum kann ein Problem nicht gelöst werden?	Reduktion auf NP-Vollständiges Problem
---	--

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

$A = (Q, \Sigma, \delta, q, F)$

Zustände: $Q = \{q_0, q_1, \dots, q_n\}$

Alphabet: Σ

Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow Q$

Startzustand: $q \in Q$

Akzeptierzustände: $F \subset Q$

0

1

Σ

q_0

q_1

q_2

q_2

q_1

q_0

q_1

q_0

q_2

δ

PUMPING LEMMA (DEA)

Um zu beweisen, dass eine Sprache **nicht** regulär ist

Vorgehen:

- 1) Annahme: L ist regulär
- 2) Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- 3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition **muss** N verwenden!
- 4) Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz, |xy| \leq N, |y| > 0$
- 5) Auswirkung des Pumpens
 - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 1 : $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- 1) Annahme: L ist regulär
- 2) $\exists N \in \mathbb{N}$, Pumping Length
- 3) $w = 0^N 1^N$
- 4) Unterteilung $w = xyz$
- 5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt
- 6) $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

Beispiel 2

TODO:

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

$A = (Q, \Sigma, \delta, q, F)$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$
- Startzustand: $q \in Q$
- Akzeptierzustände: $F \subset Q$

TODO:
Thompson-NEA

TODO:

RegEx Mengenoperationen

KONTEXTFREIE SPRACHEN (CFL)

$G = (V, \Sigma, R, S)$

- V: Variablen
- Σ : Terminalsymbole (Alphabet)
- R: Regeln der Form $A \rightarrow x_1 x_2 \dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$
- S: Startvariable

TODO:
parse-tree

PUMPING LEMMA (CFL)

Um zu beweisen, dass eine Sprache **nicht** kontextfrei ist

Voraussetzungen:

- 1) $|vxy| > 0$
- 2) $|vxy| \leq N$
- 3) $uv^kxy^kz \in L \forall k \in \mathbb{N}$

Beispiel 3

TODO:

CHOMSKY-NORMALFORM (CNF)

- S kommt auf der rechten Seite nicht vor
- Jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$
- $S \rightarrow \epsilon$ ist erlaubt

Beispiel 4

TODO:

STACKAUTOMAT (PDA)

$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- 1) Q: Zustände
- 2) Σ : Eingabe-Alphabet
- 3) Γ : Stack-Alphabet
- 4) $\delta: Q \times \Sigma_{\epsilon} \times \Gamma_{\epsilon} \rightarrow P(Q \times \Gamma_{\epsilon})$
- 5) $q_0 \in Q$: Startzustand
- 6) $F \subset Q$: Akzeptierzustände

- a vom Input
- b vom Stack entfernen (Bedingung)

c auf den Stack

TURING MASCHINEN

(deterministische) Turing-Maschine
 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

- Q: Zustände
- Σ : Alphabet
- Γ : Bandalphabet, $\sqcup \in \Gamma \setminus \Sigma$
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- $q_0 \in Q$: Startzustand
- $q_{\text{accept}} \in Q$: Akzeptierzustand
- $q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

- Übergang möglich, wenn a unter dem Schreib- / Lese-Kopf
- Aktuelles Feld auf dem Band wird mit b überschrieben
- Kopfbewegung: L links, R rechts

TODO:
berechnungsgeschichte

TODO:
varianten

ENTSCHEIDBARKEIT

TODO:

Entscheidbare Probleme

SATZ VON RICE

Ist P eine nichttriviale Eigenschaft Turing-erkennbarer Sprachen, dann ist P_{TM} nicht entscheidbar.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

LÖSUNGSANLEITUNG EINER PRÜFUNGSFRAGE:

- Nichttriviale Eigenschaft P aufschreiben
- Die beiden Sprachen $L(M_1)$ und $L(M_2)$ bilden (meistens kann man die leere Menge oder Σ^* als eine dieser Sprachen verwenden)
- Gibt es ein Programm, welches beide Sprachen erkennen kann? Sind beide Sprachen Turing erkennbar?
- Dann besagt der Satz von Rice, dass die Sprache nicht entscheidbar ist

TODO:

Beispiel

NP

TODO:

POLYNOMIELLER VERIFIZIERER

TODO:

NURIKABE

Entscheidbar? Ja: $n = uv$ = Anzahl Felder. Alle Belegungen des Spielfeldes mit schwarzen Feldern können überprüft werden, ob sie die Regeln einhalten.

Zertifikat: Liste der schwarzen Felder

Vorgehen:

Schritt	Aufwand
1 Für jedes Zahlfeld ($O(n)$) verwendet man einen Markieralgorithmus, der alle zum Gebiet dieses Zahlfeldes gehörenden weissen Felder bestimmt ($O(n)$ Durchgänge mit Aufwand $O(n)$).	$O(n^3)$
2 Trifft dieser Algorithmus auf ein weiteres Zahlfeld, ist der erste Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$O(n)$
3 Weicht die Zahl der gefundenen weissen Felder vom Inhalt des Zahlfeldes ab, ist der zweite Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$n \cdot O(n)$
4 Ebenfalls mit einem Markieralgorithmus wird überprüft, ob das schwarze Gebiet zusammenhängend ist.	$O(n^2)$
5 Für jedes schwarze Feld wird überprüft, ob es Teil eines 2×2 -Tümpels ist.	$O(n)$
Total	$O(n^3)$

REDUKTION

TODO:

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

Zu beachten (Punkteverteilung):

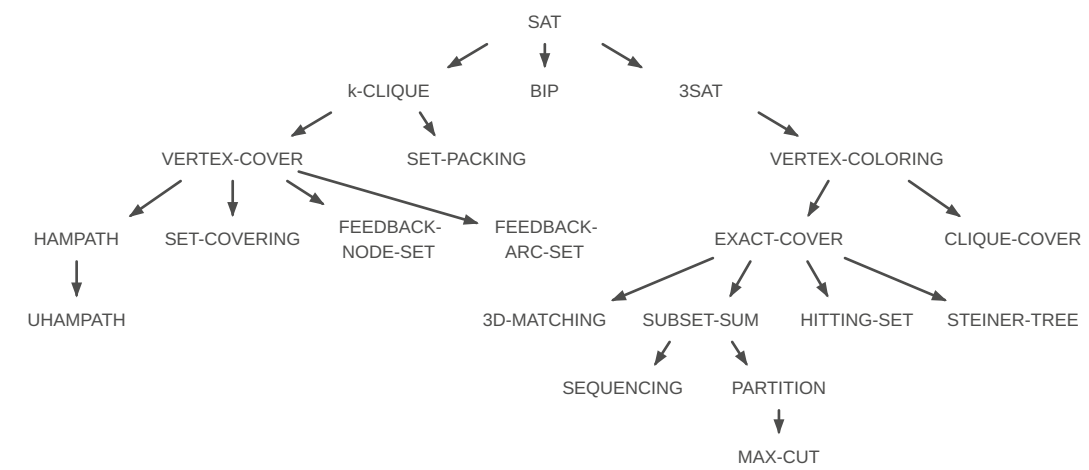
- Lösungsprinzip der Reduktion erwähnen (1P)
- Auswahl eines geeigneten Vergleichsproblems (1P)
- Reduktionsabbildung (*P)
- Schlussfolgerung: NP-Vollständig und somit nicht effizient lösbar (1P)

TODO:

AutoSpr | FS26

Seite 1

KARP-KATALOG	MENGEN	AUFTEILUNG IN ZWEI MENGEN	PFAD E IN GRAPHEN	LOGIK
PROBLEME MIT k ZAHLEN		PARTITION		SAT
VERTEX-COLORING	CLIQUE-COVER	Gibt es eine Unterteilung einer Liste S von ganzen Zahlen in zwei disjunkte Listen mit gleicher Summe?	Ein hamiltonscher Pfad in einem Graphen $G = (V, E)$ ist ein Pfad, der jeden Knoten des Graphen genau einmal besucht. Ein solcher Pfad definiert eine Reihenfolge $a_1, \dots, a_n$ von Knoten, so dass jede Kante von a_i nach a_{i+1} in E ist. Für einen gerichteten Graphen sind dies die Kanten $(a_i, a_{i+1}) \in E$, für einen ungerichteten Graphen gilt $\{a_i, a_{i+1}\} \in E$. Ein hamiltonscher Zyklus ist ein geschlossener hamiltonscher Pfad.	Gegeben einer aussagenlogischen Formel, gibt es eine Zusammensetzung der Variablenwerte, die die Aussage «Wahr» werden lassen?
EXACT-CLIQUE-COVER	K-CLIQUE	Beispiel: Unterteilung der Prominenten + Entourage auf zwei Säle des Film-Festivals.	HAMPATH	Beispiel: Elektriker: Konfiguration der Schalter zweier Stromkreise, die in allen Räumen Licht brennen lassen sollten
VERTEX-COVER	SET-PACKING	Lösung: $i \in I = \text{Promi}, c_i = \text{Anz. Mitglieder seiner Entourage}, A = \text{Stars samt Entourage für Saal A}, B = \text{für Saal B}, I = A \cup B, \sum_{i \in A} c_i = \sum_{i \in B} c_i$	Gibt es einen gerichteten hamiltonschen Pfad im gerichteten Graphen G ?	Lösung: $x_i = \text{Schalter}, T \vee F = \text{Stromkreise (werte der Variable } x_i, \text{ Lampen des ersten Kreises leuchten, wenn } x_i = T), x_{i_1} \vee \dots \vee x_{i_k} = \text{ob Licht im Raum der mit } x_i \text{ verbundenen Schalter brennt}$
SET-COVERING	MAX-CUT	MAX-CUT	Gibt es ganze Schweiz bereisen, ohne eine Stadt mehrmals zu besuchen	3SAT
STEINER-TREE	SEQUENCING	Gegeben ein Graph $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$ der Kanten und einer ganzen Zahl $k \in \mathbb{Z}$, gibt es eine Aufteilung der Knotenmenge $V = V_1 \cup V_2$ in zwei disjunkte Teilmengen, so dass das Gewicht der Kanten, die V_1 und V_2 verbinden, mindestens w ist:	Lösung: $G = \text{Linienetz}, V = \text{Stadt}, E = \text{ÖV-Verbindung}, \text{Hamiltonscher Pfad} = \text{Reise}$	Wie SAT, nur hat jede Klausel der Normalform maximal 3 Literale.
FEEDBACK-NODE-SET	FEEDBACK-ARC-SET	$\varphi(V_1, V_2) = \sum_{v_1 \in V_1, v_2 \in V_2, e = \{v_1, v_2\} \in E} \varphi(e) \geq k$?	HAMCIRCUIT	Beispiel: SAT $\rightarrow$ 3SAT
GRAPH-ÜBERDECKUNGEN		KOMBINATORISCHE OPTIMIERUNG	Wie HAMPATH, aber zusätzlich muss der Pfad geschlossen sein	Lösung:
Handeln davon, dass ein gegebener Graph von einer Menge beschränkter Größe mit einer besonderen Eigenschaft aus erreicht werden kann.		Handelt um das finden eines Maximums oder Minimums einer Zielfunktion.	UHAMPATH	$(x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (x_5 \vee x_6)$ $\rightarrow (x_1 \vee x_2 \vee z_1) \wedge (\bar{z}_1 \vee x_3 \vee x_4) \wedge (x_5 \vee x_6 \vee x_6)$
VERTEX-COLORING		STEINER-TREE	Wie UHAMPATH, aber ungerichtet	WEITERE
Gegeben ist ein Graph $G = (V, E)$ mit Knoten V und Kanten K und eine Zahl k . Ist es möglich, die Knoten des Graphen mit k Farben so einzufärben, dass durch Kanten verbundene Knoten verschiedene Farben haben?		Gibt es zu einem Graphen $G = (V, E)$ mit einer Gewichtsfunktion $\varphi: E \rightarrow \mathbb{Z}$, einer Teilmenge $R \subset V$ der Knoten und einer Zahl k einen Teilbaum $T = (V_1, E_1) \subset G$ des Graphen, der alle Knoten von R enthält und dessen Kantengewicht $\sum_{e \in E_1} \varphi(e) \leq k$ ist?	UHAMCIRCUIT	SUBSET-SUM
Beispiel: Stundenplan: Planung der Fächer		Beispiel: Feindliche Übernahme einer Firma, mit resultierenden Aufteilung der Abteilung, sodass diese möglichst ineffizient miteinander kommunizieren können.	Wie UHAMPATH, aber zusätzlich muss der Pfad geschlossen sein	Gegeben einer Liste S von natürlichen Zahlen $s_i \in \mathbb{N}$ und einer natürlichen Zahl $t \in \mathbb{N}$, ist es möglich eine teilste $T \subset S$ auszuwählen, sodass die Elemente von T dies Summe $t = \sum_{s \in T} s$ haben?
$f \in F$ auf k Zeitfenster, damit keine Anmeldungen $\{f_1, f_2\} \in A$ in gleichen Zeitfenstern liegen		Lösung: $V = \text{Abteilung}, E = \text{Kommunikationsbeziehung}, \varphi = \text{Kommunikationsvolumen}$		Beispiel: Rucksack-Problem (Ziel: Rucksack füllen)
Lösung: Knoten $V = \text{Fach}, \text{Kanten } E = \text{Anmeldungen}, \text{Farben} = \text{Zeitfenster}$		SEQUENCING		Lösung: $S = \text{Größen der Gegenstände}, t = \text{Max. Platz im Rucksack}, T \subset S = \text{Größen der Gegenstände}, \text{die in den Rucksack passen}$
CLIQUE-COVER				3D-MATCHING
Gibt es k Cliques so, dass jede Ecke in genau einer der Cliques ist?				Gegeben sind drei endliche Mengen X, Y und Z mit gleich vielen $n = X = Y = Z $ Elementen und $T \subset X \times Y \times Z$. Gibt es eine n -elementige Teilmenge $M \subset T$ derart, dass keine zwei Tripel von M in einer Koordinate übereinstimmen?
CLIQUE-COVER				Beispiel: Ein Koch hat n Rezepte für Vorspeisen, Hauptspeisen und Desserts. Nicht alle Vorspeisen lassen sich mit jeder Hauptspeise kombinieren, dasselbe gilt auch für Desserts. Er möchte n Menus zusammenstellen, sodass jedes Rezept in genau einem der Menus vorkommt.
$k = 3$				Lösung: $X = \text{Vorspeisen}, Y = \text{Hauptspeisen}, Z = \text{Desserts}, T \subset X \times Y \times Z = \text{Menge aller Menus}, M \subset T = \text{Menge der zusammengestellter Menus}$
EXACT-CLIQUE-COVER				BIP
Wie CLIQUE-COVER, nur müssen die Mengen disjunkt sein.				BIP = Binary Integer Programming
Beispiel: Bilden von k Gruppen von Leuten, die sich kennen. Alle Leute sollen eingeteilt sein.				
Lösung: Knoten $V = \text{Teilnehmer}, \text{Kanten } E = \text{Kennen sich}, k = \text{Anzahl Gruppen}, \text{Clique } G_k = \text{Gruppe}$				
K-CLIQUE				
Wie CLIQUE-COVER, nur ist k bekannt.				
Beispiel: Job-Parallelisierbarkeit: Menge von n Jobs, die jeweils exklusiv auf m Ressourcen zugreifen, Jobs dürfen nicht gleichzeitig laufen. Ziel: Entscheiden, ob zu irgendeinem Zeitpunkt mehr als k Prozessoren ausgelastet werden können				
Lösung: Graph $G = \text{Job-Parallelisierbarkeit}, \text{Knoten } V = n \text{ Job}, \text{Kanten } E = m \text{ gleiche Ressourcen}, k\text{-Clique } G_k = \text{Auswahl Knoten ohne Verbindung}, k = \text{Auslastbare Anzahl Prozessoren}$				
VERTEX-COVER				
Gibt es eine Menge W von k Knoten derart, dass jede Kante von G einen Endpunkt in W hat?				
Beispiel: Kontrolle des Verkehrsnetzes: Mitarbeiter haben ihre Basis an einzelnen Knotenpunkten. Ziel: An welchen Stationen muss man Kontrolleure stationieren, damit jede Strecke bei einem Kontrolleur endet?				
Lösung: Knoten $V = \text{Stationen}, \text{Kanten } E = \text{Strecke}, k = \text{Anzahl Kontrolleure}, W \subset V = \text{Knoten mit Kontrolleur}$				



DIFFERENZMENGE $L(A_1) \setminus L(A_2)$

A_2

$A_1 \setminus A_2$

A_1

$A_1 \times A_2$

MENGEN
SCHNITTMENGE $L(A_1) \cap L(A_2)$

A_2

$A_1 \cap A_2$

A_1

$A_1 \times A_2$

SYMMETRISCHE DIFFERENZ $L(A_1) \triangle L(A_2)$

A_2

$A_1 \triangle A_2$

A_1

$A_1 \times A_2$

VEREINIGUNGSMENGE $L(A_1) \cup L(A_2)$

A_2

$A_1 \cup A_2$

A_1

$A_1 \times A_2$

AutoSpr | FS26

Seite 3